

NLP Lab Practical Code Repository

Comprehensive Compilation of Assignments (nlp1 to nlp10)

```
# Import required libraries
import nltk
import re
import string
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords

# Download required datasets
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('punkt_tab')

# Sample text
text = """
Hello! My name is siddhesh. You can contact me at siddhesh@gmail.com
or visit https://geca.com. Call me at 1231231231.
I can't attend the class today because I'm busy.
"""

print("Original Text:\n", text)

# 1. Lowercasing
text = text.lower()
print("\nAfter Lowercasing:\n", text)

# 2. Handling Contractions
contractions = {
    "can't": "cannot",
    "won't": "will not",
    "i'm": "i am",
    "it's": "it is",
    "you're": "you are",
    "don't": "do not"
}

for key, value in contractions.items():
    text = text.replace(key, value)
print("\nAfter Expanding Contractions:\n", text)

# 3. Extract data using Regex
emails = re.findall(r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-z]{2,}', text)
phones = re.findall(r'\b\d{10}\b', text)
urls = re.findall(r'https?://\S+|www\.\S+', text)
print("\nExtracted Emails:", emails)
print("Extracted Phone Numbers:", phones)
print("Extracted URLs:", urls)
```

```
# 4. Remove punctuation
text_no_punct = text.translate(str.maketrans('', '', string.punctuation))
print("\nAfter Removing Punctuation:\n", text_no_punct)

# 5. Tokenization
words = word_tokenize(text_no_punct)

# 6. Stopword Removal
stop_words = set(stopwords.words('english'))
filtered_words = [word for word in words if word not in stop_words]

print("\nFinal Processed Words:\n", filtered_words)
```

```
import nltk
import spacy
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer , WordNetLemmatizer

nltk.download('punkt')
nltk.download('wordnet')
nltk.download('punkt_tab')

text = "I am running and eating at the same time. I have better habits now."

# --- NLTK ---
tokens = word_tokenize(text)

stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()

stemmed = [stemmer.stem(word) for word in tokens]
lemmatized = [lemmatizer.lemmatize(word) for word in tokens]

print("NLTK Tokens:", tokens)
print("Stemmed:", stemmed)
print("Lemmatized:", lemmatized)

# --- spaCy ---
nlp = spacy.load("en_core_web_sm")
doc = nlp(text)

spacy_tokens = [token.text for token in doc]
spacy_lemmas = [token.lemma_ for token in doc]

print("spaCy Tokens:", spacy_tokens)
print("spaCy Lemmas:", spacy_lemmas)
```

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import SVC

texts = [
    "Exclusive deal available for ronit",
    "Hello ronit how are you",
    "Limited time offer for ronit",
    "Let's meet tomorrow ronit"
]

labels = [1, 0, 1, 0]

print("Dataset:", texts)

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(texts)

X_train, X_test, y_train, y_test = train_test_split(X, labels, test_size=0.2)

nb = MultinomialNB()
nb.fit(X_train, y_train)
print("NB Accuracy:", nb.score(X_test, y_test))

svm = SVC()
svm.fit(X_train, y_train)
print("SVM Accuracy:", svm.score(X_test, y_test))

# --- Additional Test Box ---
texts = ["Win money now", "Hello friend", "Free prize", "Let's meet tomorrow"]
labels = [1, 0, 1, 0] # 1 = spam, 0 = not spam

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(texts)

X_train, X_test, y_train, y_test = train_test_split(X, labels, test_size=0.2)

# Naive Bayes
nb = MultinomialNB()
nb.fit(X_train, y_train)
print("NB Accuracy:", nb.score(X_test, y_test))

# SVM
```

```

svm = SVC()
svm.fit(X_train, y_train)
print("SVM Accuracy:", svm.score(X_test, y_test))

# --- Bag of Words Matrix Analysis ---
bow_vectorizer = CountVectorizer()
X_bow = bow_vectorizer.fit_transform(texts)

print("Vocabulary (words and their indices):")
print(bow_vectorizer.vocabulary_)

print("\nBag of Words matrix (sparse):")
print(X_bow.toarray())
print("\nShape of BoW matrix:", X_bow.shape)

# --- TF-IDF Matrix Analysis ---
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf_vectorizer = TfidfVectorizer()
X_tfidf = tfidf_vectorizer.fit_transform(texts)

print("Vocabulary (words and their indices):")
print(tfidf_vectorizer.vocabulary_)

print("\nTF-IDF matrix (sparse):")
print(X_tfidf.toarray())
print("\nShape of TF-IDF matrix:", X_tfidf.shape)

```

```
import random
import nltk
from nltk.util import ngrams

nltk.download('punkt')
nltk.download('punkt_tab')

text = "Learning Ai was a better expereince than ever, This has been a great journey
"
tokens = nltk.word_tokenize(text)

# Generate n-grams
unigrams = list(ngrams(tokens, 1))
bigrams = list(ngrams(tokens, 2))
trigrams = list(ngrams(tokens, 3))

print("Bigrams:", bigrams)
print("Trigrams:", trigrams)

# Simple text generation (bigram model)
def generate_text(bigrams, start_word, length=10):
    current = start_word
    result = [current]

    for _ in range(length):
        candidates = [b[1] for b in bigrams if b[0] == current]
        if not candidates:
            break
        current = random.choice(candidates)
        result.append(current)

    return ' '.join(result)

print("Generated Text:", generate_text(bigrams, "I"))
```

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix
import pandas as pd

# Naive Bayes and SVM Evaluation Block
nb = MultinomialNB()
nb.fit(X_train_vectorized, y_train_data)
y_pred_nb = nb.predict(X_test_vectorized)
print("Naive Bayes Classifier:")
print(f"Accuracy: {nb.score(X_test_vectorized, y_test_data) * 100:.2f}%")
print("Classification Report:")
print(classification_report(y_test_data, y_pred_nb))
print("Confusion Matrix:\n", confusion_matrix(y_test_data, y_pred_nb))

svm = SVC(random_state=42)
svm.fit(X_train_vectorized, y_train_data)
y_pred_svm = svm.predict(X_test_vectorized)
print("\nSupport Vector Machine Classifier:")
print(f"Accuracy: {svm.score(X_test_vectorized, y_test_data) * 100:.2f}%")
print("Classification Report:")
print(classification_report(y_test_data, y_pred_svm))
print("Confusion Matrix:\n", confusion_matrix(y_test_data, y_pred_svm))

# --- Dataset Fetch & Preprocessing Block ---
# !wget -nc https://archive.ics.uci.edu/ml/machine-learning-databases/00228/
# smsspamcollection.zip
# !unzip -n smsspamcollection.zip

# Load the dataset
df = pd.read_csv('SMSSpamCollection', sep='\t', header=None, names=['label',
'message'])

# Display dataframe info
print(df.head())
print(df.info())
print(df['label'].value_counts())

df['label'] = df['label'].map({'ham': 0, 'spam': 1})

X_data = df['message']
y_data = df['label']
```



```
X_train_raw, X_test_raw, y_train_data, y_test_data = train_test_split(
    X_data, y_data, test_size=0.2, random_state=42, stratify=y_data
)

vectorizer = CountVectorizer()
X_train_vectorized = vectorizer.fit_transform(X_train_raw)
X_test_vectorized = vectorizer.transform(X_test_raw)

print("Data preprocessing complete. Ready for model training.")
```

```
!pip install gensim

from gensim.models import Word2Vec
import numpy as np

sentences = [["event", "was", "sensational"], ["computer vision", "is",
"incredible"]]

# Word2Vec Model
model = Word2Vec(sentences, vector_size=100, window=5, min_count=1)
print("Word2Vec vector for 'event':", model.wv['event'])

# Function to load GloVe Embeddings
def load_glove(file):
    embeddings = {}
    with open(file, encoding='utf8') as f:
        for line in f:
            values = line.split()
            word = values[0]
            vector = np.array(values[1:], dtype='float32')
            embeddings[word] = vector
    return embeddings
```

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report
import random
```

```
# Core Text Dataset
```

```
texts = [
    "I love this product, it's fantastic!",
    "This is absolutely terrible, a waste of money.",
    "The service was amazing, very helpful.",
    "Worst experience ever, I'm so disappointed.",
    "Highly recommend, really good quality.",
    "Not happy with my purchase, it broke quickly.",
    "Excellent functionality, works as expected.",
    "Poor design, very difficult to use.",
    "A truly positive interaction.",
    "Completely negative and frustrating.",
    "This movie was captivating and brilliantly acted.",
    "I regret buying this; it's a complete rip-off.",
    "The food was delicious and the ambiance perfect.",
    "What a terrible experience from start to finish.",
    "So impressed with the speed and efficiency!",
    "Absolutely awful, I wouldn't recommend it to anyone.",
    "A truly wonderful and delightful surprise.",
    "Utterly useless and a massive disappointment.",
    "Fantastic customer support, they resolved my issue quickly.",
    "My worst purchase this year, very poor quality.",
    "This is a must-have, an outstanding innovation.",
    "I despise everything about this item, horrible.",
    "Very satisfied with my recent purchase.",
    "Extremely disappointed with the lack of features.",
    "The best software I've used in a long time.",
    "Such a frustrating interface, hard to navigate.",
    "Highly enjoyable and worth every penny.",
    "A complete scam, avoid at all costs.",
    "Pleasant surprise, exceeded my expectations.",
    "An absolute disaster, totally unacceptable.",
    "Absolutely loving this, it's a game-changer!",
    "This is mediocre at best, not worth the hype.",
    "Customer support was stellar, truly exceptional.",
    "Beyond terrible, I want my money back immediately.",
    "An amazing experience from beginning to end.",
    "Disappointing quality, fell apart after a week.",
```

```

"Works perfectly, exactly what I needed.",
"Flawed design, constant issues.",
"Such a delightful product, highly recommend.",
"A total ripoff, stay away from this.",
"The performance is outstanding, very impressed.",
"Horrible customer service, very rude and unhelpful.",
"Couldn't be happier with my purchase, 5 stars!",
"Worst purchase ever, seriously disappointed.",
"This will change your life, incredible!",
"Broken on arrival, absolute rubbish.",
"A joyous and pleasant surprise.",
"Waste of time and money, completely ineffective.",
"They went above and beyond to help me.",
"I deeply regret buying this, a total failure.",
"Innovatively designed, works flawlessly.",
"Such a frustrating interface, hard to navigate.",
"Highly enjoyable and worth every penny.",
"A complete scam, avoid at all costs.",
"Pleasant surprise, exceeded my expectations.",
"An absolute disaster, totally unacceptable.",
"Truly inspiring and well-made.",
"Not what I expected, very poor.",
"The setup was simple and intuitive.",
"Glitches constantly, unusable.",
"A fantastic investment.",
"Overpriced and underperforming.",
"Great value for the money.",
"Doesn't work as advertised, misleading.",
"My new favorite gadget!",
"Such a negative experience.",
"Highly satisfied with the durability.",
"Broke after first use, fragile.",
"A refreshing and unique product.",
"Pure junk, do not buy."
]

labels = [
    1, 0, 1, 0, 1, 0, 1, 0, 1, 0,
    1, 0, 1, 0, 1, 0, 1, 0, 1, 0,
    1, 0, 1, 0, 1, 0, 1, 0, 1, 0,
    1, 0, 1, 0, 1, 0, 1, 0, 1, 0,
    1, 0, 1, 0, 1, 0, 1, 0, 1, 0,
    1, 0, 1, 0, 1, 0, 1, 0, 1, 0,
    1, 0, 1, 0, 1, 0, 1, 0, 1, 0
]

label_map = {0: 'Negative', 1: 'Positive'}

# Model training block

```

```

X_train_text, X_test_text, y_train, y_test = train_test_split(texts, labels,
test_size=0.3, random_state=42)
vectorizer = TfidfVectorizer(ngram_range=(1, 2))
X_train = vectorizer.fit_transform(X_train_text)
X_test = vectorizer.transform(X_test_text)

model = MultinomialNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print(f"Model Accuracy: {accuracy_score(y_test, y_pred):.2f}")
print("Classification Report:\n", classification_report(y_test, y_pred,
target_names=[label_map[0], label_map[1]]))

# --- Synthetic Review Generator Block ---
positive_words = [
    "excellent", "fantastic", "amazing", "wonderful", "great", "superb",
    "love it", "highly recommend", "perfect", "brilliant", "outstanding",
    "delightful", "impressive", "top-notch", "superior", "joyful",
    "efficient", "effective", "user-friendly", "seamless", "smooth"
]
negative_words = [
    "terrible", "awful", "horrible", "worst", "disappointing", "bad",
    "waste of money", "not happy", "broke quickly", "poor design", "difficult",
    "frustrating", "useless", "regret buying", "scam", "ripoff"
]

def generate_synthetic_review(sentiment_type):
    if sentiment_type == 'positive':
        words = random.sample(positive_words, random.randint(3, 6))
        template = "Overall, a very {} experience. This product is {}."
    else:
        words = random.sample(negative_words, random.randint(3, 6))
        template = "I find this to be {}. Extremely {} design."
    return template.format(', '.join(words[:-1]), words[-1])

# Generate larger synthetic dataset
new_texts, new_labels = [], []
for _ in range(500):
    new_texts.append(generate_synthetic_review('positive'))
    new_labels.append(1)
    new_texts.append(generate_synthetic_review('negative'))
    new_labels.append(0)

print(f"Generated {len(new_texts)} synthetic reviews.")

```

```
from sklearn.decomposition import LatentDirichletAllocation
from sklearn.feature_extraction.text import CountVectorizer

docs = [
    "he studies in GECA",
    "GECA is an engineering college",
    "Politics and government",
    "Elections and democracy",
    "Artificial intelligence and machine learning",
    "Deep learning for image processing",
    "Cricket and football are popular sports",
    "Health and fitness are important",
    "Python programming for data science",
    "Cloud computing and cybersecurity"
]

# Convert text into matrix
vectorizer = CountVectorizer(stop_words='english')
X = vectorizer.fit_transform(docs)

# Create LDA Model with 10 Topics
lda = LatentDirichletAllocation(
    n_components=10,
    random_state=42
)

# Train model
lda.fit(X)

# Get feature names
terms = vectorizer.get_feature_names_out()

# Display Topics
for topic_index, topic in enumerate(lda.components_):
    print(f"\nTopic {topic_index + 1}:")
    top_words = [terms[i] for i in topic.argsort()[-5:]]
    print(top_words)
```

```
from transformers import AutoTokenizer, AutoModelForQuestionAnswering
import torch

MODEL_NAME = "distilbert-base-cased-distilled-squad"

# Load model only ONCE
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = AutoModelForQuestionAnswering.from_pretrained(MODEL_NAME)
model.eval()

def answer_question(question: str, context: str) -> str:
    inputs = tokenizer(
        question,
        context,
        return_tensors="pt",
        truncation=True,
        max_length=256
    )

    with torch.no_grad():
        outputs = model(**inputs)

    start_idx = torch.argmax(outputs.start_logits)
    end_idx = torch.argmax(outputs.end_logits) + 1

    answer = tokenizer.decode(
        inputs["input_ids"][0][start_idx:end_idx],
        skip_special_tokens=True
    )
    return answer

context = """
Nowadays banking has become highly advanced due to modern technology.
Customers can transfer money online, check account balances using mobile apps,
and pay bills instantly. Banks use artificial intelligence for fraud detection
and customer support. Internet banking allows users to access their accounts
anytime and anywhere. Digital payments and cashless transactions have made
banking faster, safer, and easier for everyone.
"""

question1 = "What can customers do using mobile banking apps?"
print("Question:", question1)
print("Answer:", answer_question(question1, context))
```

```
question2 = "Why do banks use artificial intelligence?"  
print("Question:", question2)  
print("Answer:", answer_question(question2, context))
```



```
import nltk
import heapq
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize, sent_tokenize
from summa.summarizer import summarize

nltk.download('punkt')
nltk.download('stopwords')
nltk.download('punkt_tab')

text = """
Artificial Intelligence is rapidly transforming industries around the globe.
It allows machines to learn from large amounts of data and make intelligent
decisions.
AI is widely applied in healthcare, finance, education, and other sectors.
At the same time, it introduces challenges related to privacy, ethics, and job
displacement.
Many organizations are investing heavily in AI research and development.
"""

# --- Frequency-Based Extractive Summary ---
stop_words = set(stopwords.words("english"))
words = word_tokenize(text.lower())
freq_table = {}
for word in words:
    if word not in stop_words:
        freq_table[word] = freq_table.get(word, 0) + 1

sentences = sent_tokenize(text)
sentence_scores = {}
for sent in sentences:
    for word in word_tokenize(sent.lower()):
        if word in freq_table:
            sentence_scores[sent] = sentence_scores.get(sent, 0) + freq_table[word]

summary_sentences = heapq.nlargest(2, sentence_scores, key=sentence_scores.get)
extractive_summary = " ".join(summary_sentences)
print("Extractive Summary:")
print(extractive_summary)

# --- TextRank Summary using Summa ---
textrank_summary = summarize(text, ratio=0.5)
```

```
print("\nExtractive Summary (TextRank):")  
print(textrank_summary)
```